Hangman Game – Group Assignment

A paper submitted in partial fulfillment of the requirements for the course Management Science and Operations Research

Kühne Logistics University

Students:

Daria Fedorova (990215083)

Marcela Santos (990215807)

Simran Singh (990215792)

Selvi Lesmana Putri (990215892)

Supervisor: Prof. Dr. Arne Heinold

Date of Submission: March 09, 2026

GitHub Link: singh2894/Hangman

Table of Contents

AI USE STATEMENT 3
I. INTRODUCTION 3
II. GAME DESCRIPTION: THE HANGMAN GAME 3
III. MARKOV DECISION PROCESS FORMULATION 4
IV. ENVIRONMENT SETUP 5
V. HEURISTIC APPROACH: RULE-BASED STRATEGY 9
VI. REINFORCEMENT LEARNING (Q-LEARNING & SARSA) 10
VII. COMPARISON OF ALGORITHMS 14
VIII. CONCLUSION 15
IX. RETROSPECTIVE 15

AI Use Statement

Artificial intelligence (AI) tools were used in the preparation of this project for the following purposes: (1) code suggestions and debugging assistance during implementation of the Gymnasium environment and Q-learning agent; (2) grammar and clarity review of written sections; and (3) generation of comparative performance commentary. All algorithmic design decisions, mathematical formulations, experimental configurations, and interpretations presented in this report are the original work of the author. The use of AI tools is acknowledged in accordance with academic integrity guidelines.

I. Introduction

The Hangman game can be interpreted as a two-player sequential decision game. In this setting, Player 1 is an agent whose objective is to identify a hidden word, while Player 2 represents the environment, which selects a word from a predefined dictionary and reveals information about it as the game progresses. The agent attempts to uncover the word by sequentially guessing letters from the English alphabet. After each guess, the environment provides feedback by revealing whether the letter appears in the word and, if so, in which positions.

Although the rules of Hangman are simple, the game represents a structured decision-making problem. The agent does not know the true word and can only observe the currently revealed pattern of letters along with the letters that have already been guessed. At each step, the agent must decide which letter to select next based on the available information. Incorrect guesses reduce the number of remaining attempts, creating a constraint that limits how much exploration the agent can perform. As a result, each decision affects both the information available in future states and the probability of successfully identifying the word.

In this project, the Hangman game is modeled as a finite episodic Markov Decision Process (MDP), where the agent interacts with the environment by selecting actions in the form of letter guesses. Within this framework, we analyze and compare three different approaches for determining the agent's actions. The first approach is a rule-based heuristic strategy that uses statistical information from the dictionary to guide letter selection. The second and third approaches are reinforcement learning algorithms, specifically Q-Learning and SARSA, which attempt to learn effective strategies through repeated interaction with the environment.

The objective of this study is to evaluate whether reinforcement learning methods can learn an effective strategy for solving Hangman and to compare their performance with a heuristic approach that relies on predefined rules.

II. Game Description: The Hangman Game

The Hangman game considered in this study is a sequential word-guessing task in which an agent attempts to identify a hidden word under a limited number of incorrect guesses. At the beginning of each

episode, a five-letter English word is randomly selected from a predefined dictionary and represented by five blank positions (e.g., "_ _ _ _ _"). The agent interacts with the environment by sequentially selecting letters from the English alphabet in order to reveal the hidden word.

The agent is allowed a maximum of six incorrect guesses. At each step, the agent selects a letter as an action. If the selected letter appears in the hidden word, all corresponding positions are revealed while the number of remaining guesses remains unchanged. If the letter does not appear in the word, the number of remaining attempts decreases by one. Letters that have already been guessed cannot be selected again.

The episode ends when the word is fully revealed or when the number of remaining guesses reaches zero. The agent's objective is to maximize cumulative reward by revealing correct letters and completing the word before exhausting the allowed number of attempts. Because each guess changes the available information and influences future decisions, the Hangman game can be naturally modeled as a sequential decision-making problem within the framework of a Markov Decision Process.

III. Markov Decision Process Formulation

To formally describe the Hangman game as a sequential decision problem, it is modeled as a **Markov Decision Process (MDP)**. Formally, an MDP is defined as M=(S,A,P,R), where S represents the set of states, A the set of actions, P(s′|s,a) the transition function, and R(s,a) the reward function.

States

A state describes the current situation of the game. In the Hangman environment the state can be written as s=(p,G,t), where p denotes the current word pattern (e.g., "_a__e"), G represents the set of guessed letters, and t is the number of remaining turns. Since the hidden word consists of five letters, the number of possible word combinations can be approximated as

$$26^5 \approx 11.9\, million$$

Actions

At each step, the agent selects a letter from the English alphabet. The action space is therefore defined as A={a,b,c,…,z}, which contains 26 possible actions. Because previously guessed letters cannot be chosen again, the set of valid actions in a state s is A(s)=A\G.

Outcomes

Each action leads to one of several possible outcomes. At the level of a full episode, the game ends either with the agent winning or losing: {Agent Won, Agent Lost}. During each step, three outcomes may occur: a correct new letter, a wrong new letter, or a repeated letter.

Rewards

The reward function provides feedback after each action. A correct guess yields a positive reward, while an incorrect guess results in a penalty. The reward function is defined as:

$$\begin{cases} +1 & \text{if the guessed letter is correct} \\ -1 & \text{if the guessed letter is incorrect} \end{cases}$$

Additionally, the agent receives a bonus reward when the word is successfully completed:

$$R_{win} = +5$$

Transition Model or Dynamic Function

The transition function determines how the state changes after an action is taken. If the guessed letter appears in the word, the pattern is updated and the number of remaining attempts remains unchanged. If the letter is incorrect, the number of remaining attempts decreases by one. The transition can be expressed as: $s_{t+1} = f(s_t, a_t)$.

The transition process of the Hangman environment is illustrated in **Figure 1**, which shows how the number of remaining guesses evolves and how the game eventually reaches either a win or a loss state.

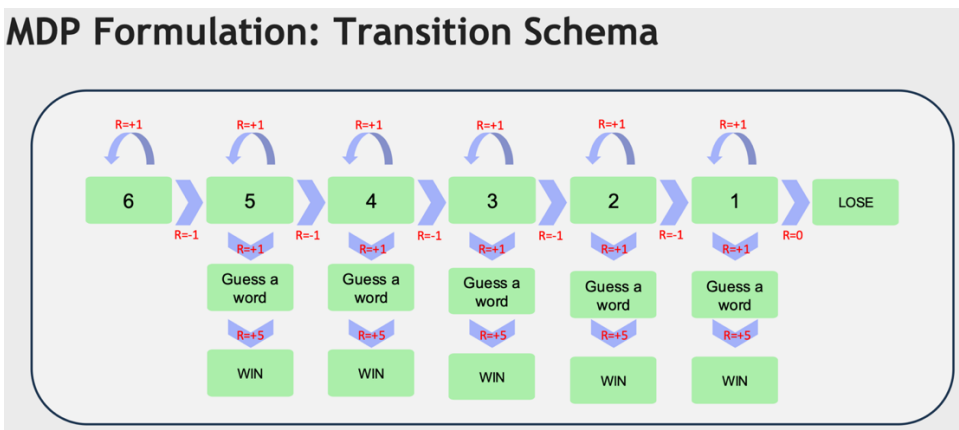


Figure 1. Hangman Game Transition Schema

IV. Environment Setup

To formalize Hangman as a reinforcement learning problem, we require an environment that

(i) maintains the game state,

(ii) accepts agent actions,

(iii) computes state transitions and rewards, and

(iv) signals episode termination.

Rather than relying on a pre-computed state graph, which would be computationally infeasible for realistic vocabulary sizes, we implement a real-time transition model that computes successor states as actions are taken.

The environment is implemented in a single Python module consisting of global constants, a Word Sampler for vocabulary management, the apply action transition function, EnvHangman as the base environment, EnvHangmanGym as a Gymnasium-compliant wrapper, and Policy Random as a baseline agent.

Constants and Configuration

The environment defines two global constants. MAX_WRONG is set to 6, representing the maximum incorrect guesses before termination, consistent with the traditional Hangman game. LETTERS is a tuple of all 26 lowercase English letters, defining the full action space. Both constants are shared across environment variants.

Word Sampler: Vocabulary Management

The Word Sampler class loads, partitions, and samples words from the dictionary. Upon instantiation, it splits words into a training set (80%) and testing set (20%) using a deterministic shuffle with seed=42, ensuring reproducibility across experiments.

Why split data into reinforcement learning? The answer lies in generalization. The agent trains 80% of words and is tested on the unseen 20% to verify it has learned transferable Hangman strategy such as prioritizing vowels and common consonants rather than memorizing specific words.

The Transition Function: apply action

The apply action function is the core engine of the environment, implementing state transitions as a pure function that computes successor states without side effects.

- ***Real-Time Transitions vs. Pre-computed State Graphs***

Pre-computing the entire state graph via BFS is infeasible: it requires exponential time, millions of stored entries, and breaks with larger dictionaries. The real-time approach via apply action requires only O (word length) per step, stores only the current state, and scales to any dictionary size computing transitions only for states the agent visits.

- ***Transition Logic***

Given inputs (word, pattern, guessed, wrong left, action): the letter is added to the guessed set, then checked against the word. If present, all occurrences are revealed in the pattern with wrong left unchanged. If absent, the pattern stays the same and wrong left decrements by one. Finally, the function returns the updated (pattern, guessed, wrong left) triplet.

- ***Worked Example***

Consider the hidden word "HELLO" with the current state: pattern = ("H", "E", "_", "_", "O"), guessed = {"H", "E", "O"}, wrong left = 4.

Case 1:

The agent guesses "L" (correct). The letter "L" appears in positions 2 and 3. Both positions are revealed simultaneously, producing the updated pattern ("H", "E", "L", "L", "O"). The wrong left counter stays at 4.

Case 2:

The agent guesses "Z" (incorrect). The letter "Z" does not appear in "HELLO". The pattern remains unchanged at ("H", "E", "_", "_", "O"), and wrong left decreases to 3.

Critically, apply action is a shared utility function used by both EnvHangman and EnvHangmanGym. This ensures that both environment variants exhibit identical game dynamics, with the only difference being how observations are formatted for the agent.

EnvHangman: The Base Environment (not calling it)

EnvHangman implements the Hangman game logic in pure Python with reset () and step () methods, independent of any external framework. It uses the same transition logic, reward structure, and termination conditions as EnvHangmanGym: +1 for correct guesses, −1 for incorrect, +5-win bonus, 0 for repeats, terminating on win, loss (wrong left = 0), or truncation at 500 steps.

The key difference is that EnvHangman returns string observations via get_obs(), producing encoded state strings usable directly as Q-table dictionary keys. It also provides get_info() for supplementary state information and get_legal_actions() for the set of unguessed letters.

EnvHangmanGym: Gymnasium-Compliant Wrapper

EnvHangmanGym extends gym.Env from the Gymnasium library, wrapping the identical game logic in an interface that conforms to the industry-standard Gymnasium API. This is the environment variant used by both the training script (train_rl.py) and the evaluation script (main.py).

Several considerations motivate the use of a Gymnasium-compatible environment. Gymnasium (formerly OpenAI Gym) is the de facto interface for RL environments in both research and industry, ensuring compatibility with existing training frameworks, logging utilities, and visualization tools. Gymnasium requires explicit declaration of the observation space and action space, which enables automated environment validation. The numeric observation format is directly compatible with neural network-based methods (e.g., DQN, PPO) for future extensions. Additionally, Gymnasium's built-in seeding mechanism provides a standardized approach to reproducibility.

The observation space is defined as gym.spaces.Box(low=0, high=27, shape=(32,), dtype=np.int32). The 32-dimensional integer vector encodes the current game state numerically: positions 0–4 represent the pattern where each element is an integer for a letter (1–26) or 0 for an unrevealed position, positions 5–30 form a binary vector indicating which of the 26 letters have been guessed, and position 31 stores the remaining wrong guess count.

The action space is defined as gym.spaces.Discrete(26), where each integer 0–25 corresponds to a letter a–z. The mapping between integer actions and letter characters is handled internally by the wrapper.

An important implementation detail is how tabular Q-Learning interfaces with EnvHangmanGym. Since Q-Learning requires hashable dictionary keys, but Gymnasium returns NumPy arrays (which are not hashable), the training script defines an obs_to_key() function that converts the NumPy observation into a tuple of integers. This tuple serves as the Q-table key, rather than the encoded string that EnvHangman would provide.

Policy Random: Baseline Agent

Policy Random implements a uniform-random policy as the performance baseline. At each step, it selects a random unguessed letter. It has no memory and no learning.

The baseline acts as a sanity check if a trained agent cannot outperform it, something is fundamentally wrong and establishes a lower performance bound. The expected win rate is approximately 5–15%.

Separation of Environment and Learning

The environment contains no learning logic. It only accepts actions, computes transitions, returns observations and rewards, and signals of termination.

All intelligence resides in separate training scripts (Q-Learning and SARSA), which maintain the Q-table, select actions via ε-greedy exploration, update Q-values, and iterate over thousands of episodes. This separation ensures the environment is reusable across algorithms without modification.

Component Interaction: The Training Loop

The training script is called Word Sampler. Sample(split="train") to obtain a word, passes it to EnvHangmanGym(secret word), applying env.reset() to get the initial NumPy observation. The observation is converted to a tuple via obs_to_key() for Q-table lookup.

The agent selects an action using its ε-greedy policy, then calls env.step(action), which internally invokes apply action and returns the successor observation, reward, and termination flags. The Q-table is updated using the TD learning rule. These steps repeat until the episode terminates, and the process continues thousands of episodes until convergence.

V. Heuristic Approach: Rule-based Strategy

A rule-based heuristic strategy was implemented to provide a deterministic method for selecting letters during the Hangman game. This approach does not rely on learning from interaction with the environment. Instead, the agent follows predefined rules based on statistical properties of the dictionary in order to determine which letter should be guessed next.

The heuristic agent operates on a dictionary containing valid five-letter English words. During initialization, the dictionary is processed and the global frequency of letters across all words is calculated. Based on these counts, a ranking of letters from the most frequent to the least frequent is constructed. This ranking reflects the general distribution of letters in the dictionary and provides a fallback decision rule when limited information about the hidden word is available.

During gameplay, the agent observes the current state of the word pattern together with the list of letters that have already been guessed. The pattern indicates which positions in the word have been revealed and which remain unknown. Using this information, the dictionary is filtered to construct a set of candidate words that are consistent with the observed pattern. A word is considered a valid candidate if it satisfies two conditions. First, all revealed letters must appear in the same positions as indicated by the pattern. Second, the word must not contain any letters that have previously been guessed and identified as incorrect. This filtering procedure reduces the set of possible words that could correspond to the hidden word.

After the candidate set is obtained, the distribution of letters within these remaining words is evaluated. For each candidate word, the letters that have not yet been guessed are considered and their frequencies across the candidate set are counted. The letter with the highest frequency is then selected as the next guess. This rule prioritizes letters that appear in many candidate words and therefore have a higher probability of revealing new information about the hidden word.

In situations where the filtering step produces no valid candidate words, the agent falls back to the global frequency ranking that was computed during initialization. In this case, the next action is chosen as the most frequent letter in the dictionary that has not yet been guessed.

Overall, the heuristic strategy can be interpreted as a greedy information-driven approach. By repeatedly selecting the letter that appears most frequently among the remaining candidate words, the agent attempts to reduce the uncertainty about the hidden word as efficiently as possible. Although this method does not involve learning from experience, it exploits statistical regularities in the dictionary and provides a strong baseline strategy for the Hangman game.

VI. Reinforcement Learning (Q-Learning & SARSA)

RL Formulation and Training Setup

Hangman was modeled as a sequential decision-making problem. At each step, the agent observes the current game state, guesses one letter, receives a reward, and transitions to the next state. The objective is to learn a policy that maximizes long-run reward while still solving as many words as possible before running out of wrong guesses.

The environment state used for training contained three pieces of information: the partially revealed word pattern, the set of letters that had already been guessed, and the number of mistakes remaining. In the Gym-style environment, these elements were encoded into a 32-dimensional observation vector. The first positions represented the current word pattern, the next 26 positions represented whether each letter of the alphabet had already been guessed, and the final slot stored the remaining number of wrong guesses. This design was important because it gave the RL agent a compact but informative state representation that was consistent across episodes.

The action space contained 26 discrete actions, one per letter from a to z. A key implementation choice was to restrict action selection to legal actions only, meaning letters that had not been guessed yet. This prevented the agent from repeatedly selecting the same letter, reduced unnecessary noise during learning, and made the Q-updates more meaningful. In practice, this decision improved stability because the agent did not waste exploration budget on clearly invalid choices.

Reward Design and Why It Mattered

The reward function was deliberately kept simple: +1 for a correct guess, -1 for a wrong guess, and an additional +5 win bonus when the word was fully solved. This shaping gave the agent both local and global guidance. The +1 / -1 feedback made each individual guess informative, while the terminal bonus encouraged the agent to complete the word instead of only maximizing immediate letter rewards.

Several reward alternatives were considered during experimentation. For example, stronger penalties for losing states were tested, but they made learning less stable. The agent was still exploring heavily in early training, so large negative terminal penalties made many episodes look uniformly bad and reduced the usefulness of the learning signal. After observing that this worsened performance, the environment was restored to the simpler reward structure above. This was one of the first major lessons of the training phase: in this setting, clearer and smaller rewards worked better than aggressive shaping.

Algorithms Implemented

Q-learning. Q-learning was implemented as an off-policy method. At each step, the Q-value update used the immediate reward plus the discounted maximum Q-value of the next state. The intuition behind using Q-learning was that it learns toward the greedy optimal action, even while the behavior policy is still exploring. This made it a strong candidate for final deployment once hyperparameters were tuned properly.

SARSA. SARSA was implemented as an on-policy method. Instead of updating toward the best possible next action, it updated using the next action that was actually selected under the epsilon-greedy policy. This made SARSA more conservative during learning and, in some intermediate experiments, more stable early in training. Implementing both methods was useful because it gave a real benchmark inside the RL part of the project rather than assuming in advance that one algorithm would dominate.

Main Training Challenges and How They Were Fixed

Environment Compatibility and Action Handling

One of the first practical issues was making sure the Gym-style environment and the training loop matched perfectly. The training code expected the environment step function to return five values: observation, reward, terminated, truncated, and info. During debugging, an edge case related to already-guessed letters returned an inconsistent output format. Even though legal-action filtering meant the agent almost never selected illegal actions, this mismatch still had to be fixed because it broke environment correctness. The solution was to standardize the environment so that step() always returned the same five-element output.

State Representation Concern

A major concern was whether the guessed letters were being represented as an ordered history or as an order-invariant mask. If guess order had changed the state identity, the Q-table would have exploded unnecessarily, and many logically identical states would have been treated as different. After checking the environment carefully, the guessed letters were confirmed to be encoded as a 26-bit mask, which was the correct design choice. This was important because it meant poor performance could not be blamed on a broken state representation.

Reward Shaping Instability

At one point a stronger loss penalty was introduced in the hope that the agent would learn to avoid failure faster. In practice, this made the average reward much worse and did not improve win rate. The reason was that during early exploration the agent naturally loses many episodes, so harsh penalties overwhelmed the learning signal. The solution was to revert to the simpler reward structure (+1, -1, +5), which produced more stable learning and more interpretable results.

Exploration Schedule Sensitivity

A major challenge was finding an exploration schedule that balanced discovery and convergence. Early results with 50,000 episodes produced low win rates because the agent was either exploring too much for too long or not exploiting what it had learned effectively enough. Different epsilon end values were tested, and the results showed that the final exploration rate mattered a lot. Too much exploration late in training kept performance noisy, meanwhile too little exploration too early limited learning. The final schedule kept epsilon-greedy exploration but decayed it to a very small value, allowing the agent to become nearly greedy by the end of training.

Choosing Between Q-Learning and SARSA

Another challenge was that intermediate experiments gave mixed signals. In some runs SARSA looked better; in others Q-learning recovered later and achieved higher final performance. Instead of relying on a single lucky run, both algorithms were tuned systematically and compared across multiple seeds. This made the final choice much more defensible.

Training Length

The strongest final lesson was that the RL agent had not converged at 50,000 episodes. Performance improved substantially when training was extended to 100,000, then 200,000, and finally 500,000 episodes. The jump from low early win rates to a final 84.2%-win rate showed that the model needed long training with a small learning rate in order to stabilize.

Hyperparameter Tuning and Testing Methodology

To avoid selecting parameters arbitrarily, a structured grid search was run over three main hyperparameters: the learning rate α, the discount factor γ, and the final exploration rate ε_end. The search space was $\alpha \in \{0.01, 0.05, 0.1, 0.2, 0.3\}$, $\gamma \in \{0.90, 0.95, 0.99\}$, and $\varepsilon_end \in \{0.05, 0.02, 0.01\}$. Each configuration was trained for 50,000 episodes and evaluated on 1,000 unseen test words using five random seeds. Reporting mean ± standard deviation was essential because reinforcement learning is stochastic, and a single run can be unusually lucky or unlucky.

The sweep revealed a very clear pattern: small learning rates worked best. In both algorithms, $\alpha = 0.01$ consistently outperformed larger step sizes, while $\alpha = 0.2$ and $\alpha = 0.3$ led to very poor performance. This makes sense for Hangman because the state space is large and the reward signal is noisy; slow, stable updates are better than aggressive ones. The best mean configuration for Q-learning was $\gamma = 0.90$, $\varepsilon_end = 0.01$, $\alpha = 0.01$, with a mean win rate of 0.319 at 50,000 episodes. The best SARSA configuration was $\gamma = 0.99$, $\varepsilon_end = 0.02$, $\alpha = 0.01$, with a mean win rate of 0.307. These results showed that both algorithms were competitive, but Q-learning achieved the higher final average and was therefore selected as the final model.

Learning Behavior

Figure 2 summarizes the learning dynamics of the tuned agents across training. Multi-seed averaging was used so the plot would reflect the underlying trend rather than a single noisy run. SARSA was slightly stronger during part of the early learning phase, but Q-learning improved more sharply later and reached the higher final performance. This supported the decision to use Q-learning as the final deployed model.

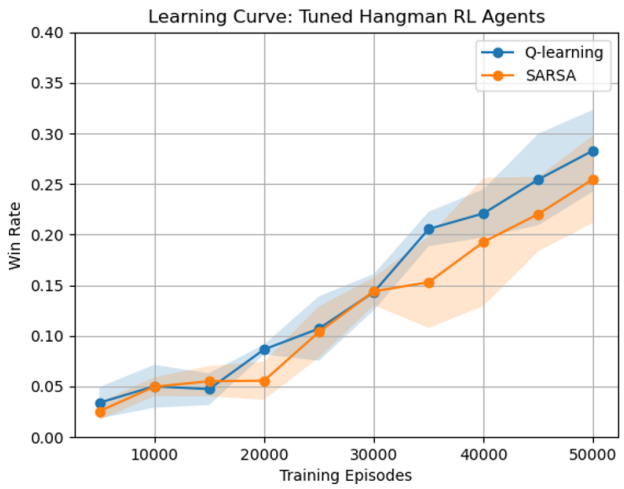


Figure 2. Learning curve of tuned Q-learning and SARSA agents during training (mean performance with variability bands across seeds).

Final Model and Results

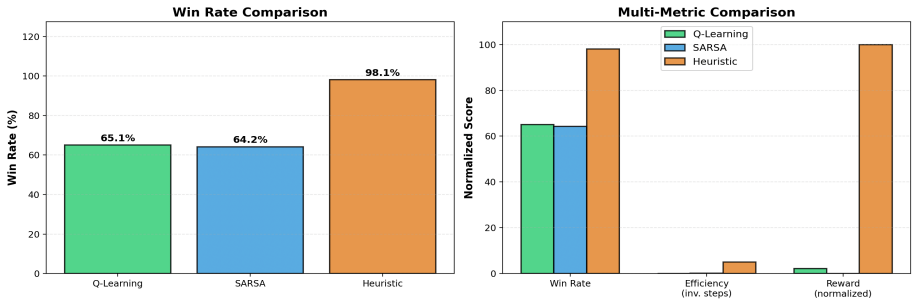
After the tuning phase, the final Q-learning model was trained with $\alpha = 0.01$, $\gamma = 0.90$, $\varepsilon_end = 0.01$ for 500,000 episodes. This long training horizon proved to be essential. Earlier checkpoints at 50,000, 100,000, and 200,000 episodes showed substantial improvement over time, which indicated that the policy was still learning and had not yet plateaued at lower episode counts.

Metric	Final value
Greedy policy win rate	0.842
Average steps per episode	7.407
Average reward per episode	5.647
States explored during training	1,356,891

The final win rate of 84.2% shows that the trained agent solves most unseen Hangman games successfully. The average of roughly 7.4 steps per episode indicates that the policy is not only accurate but also efficient. The strongly positive average reward confirms that the agent consistently makes productive guesses and frequently reaches the winning terminal state.

An important interpretation point is that this result is much stronger than the earlier 50,000-episode experiments. The documentation of the RL work is therefore not only about the final number, but about the process used to reach it: careful environment debugging, controlled hyperparameter tuning, repeated testing, and recognition that the policy required far more training than the initial baseline suggested.

VII. Comparison of Algorithms




```
Training configuration:
  Episodes: 100,000
  Evaluation games: 1,000 (test set)
Performance Comparison Table
Metric                    Q-Learning        SARSA    Heuristic
------------------------------------------------------------
Win Rate (%)                   65.10        64.20        98.10
Avg Steps per Game              7.68         7.67         7.29
Avg Reward per Game             3.74         3.67         7.01
States Explored               382975       297554          N/A
```

Figure 3. Performance Metrics Comparison of Q-Learning, SARSA, and Heuristic

After tuning both algorithms at 50,000 episodes showed Q-Learning performed better, another round of training at 100,000 episodes was conducted to confirm this result and obtain more stable performance numbers. The findings supported the earlier observations where Q-Learning won 65.1% of games compared to SARSA's 64.2%, earned average rewards of 3.74 per game versus SARSA's 3.67, and encountered 382,975 different game situations during training while SARSA encountered only 297,554. This 28.7% difference in explored states comes from how Q-Learning works where it can learn the best strategy while still trying out different approaches freely. Both algorithms took about the same number of guesses per game (7.68 for Q-Learning and 7.67 for SARSA), but Q-Learning won more often because it made better choices at important moments, not because it played faster.

The decision to stop comparing both algorithms at 100,000 episodes instead of continuing to 500,000 was made for practical reasons. Once Q-Learning showed consistent better performance during tuning and maintained that advantage through 100,000 episodes, further comparative training became unnecessary. Running both algorithms for 500,000 episodes would have required substantial computational time, and sufficient evidence already existed to select Q-Learning. Instead, computing resources were focused on training just the Q-Learning model longer which eventually reached 84.2% win rate after 500,000 episodes. This approach reflects sound research methodology where use early tests to identify which method works best, then invest effort into that one rather than spending time on the weaker option.

These results establish Q-Learning as the best reinforcement learning choice for Hangman when the agent must learn without any built-in knowledge about letters or words. The algorithm's ability to explore more situations, earn better rewards, and win more games proves that its design works well for this type of problem. The heuristic method still performs better at 98.1%, but Q-Learning's results demonstrate that reinforcement learning can reach useful performance levels with proper tuning and adequate training time. The progression from 65.1% at 100,000 episodes to 84.2% at 500,000 episodes confirms that

extended training matters significantly for approaching expert performance, which justifies the strategic choice to focus computing power on the better algorithm.

VIII. Conclusion

This project successfully formalized Hangman as a Markov Decision Process and compared three solution approaches: a rule-based heuristic, and two reinforcement learning algorithms that learn through trial and error. The heuristic achieved the highest performance at 98.1% win rate by filtering candidate words and selecting frequent unguessed letters, while Q-Learning proved most effective among learning methods, reaching 84.2% win rate after 500,000 training episodes with optimized hyperparameters compared to initial performance of 31.9% at 50,000 episodes. Several key lessons emerged during implementation: state representation required careful design to ensure equivalent game situations mapped to identical states regardless of guess order, the reward structure needed balancing between local feedback and terminal bonuses as harsh penalties hindered learning, hyperparameter tuning proved essential with small learning rates consistently outperforming larger values due to noisy reward signals and large state spaces, and systematic comparison using multiple random seeds was necessary to avoid misleading conclusions from single training runs. While domain knowledge provides substantial advantages in word games, these findings validate that tabular Q-Learning remains viable for finite discrete problems with manageable state spaces when given adequate training and proper configuration, though the gap between pure learning methods and expert strategies suggests that hybrid approaches combining both paradigms may offer the most practical path forward for real-world applications.

IX. Retrospective

Several limitations were identified during this work. Hyperparameter exploration remained limited, as the exploration rate, discount factor, and training duration were not extensively tested, which means broader tuning could potentially improve performance. The problem was restricted to 5-letter words from a 1,000-word dictionary which simplified the task but reducing how widely the results apply. Future work could explore alternative methods like dynamic programming or different exploration strategies to better balance discovery and exploitation. Expanding to longer words (6-7 letters) and larger dictionaries (2,000-5,000 words) would create more challenging conditions for testing different approaches.